



ESTRUTURA DE DADOS

PONTEIROS

Prof. Me. Gustavo Ferreira Palma

PAUTA

- Ponteiros

PONTEIROS

PONTEIROS - CONCEITO

- Toda variável declarada em um programa ocupa uma ou mais posições na memória. Essas posições são identificadas por endereços, que funcionam como "coordenadas" utilizadas pelo processador para localizar os dados.
- Um ponteiro é uma variável cujo conteúdo não é um valor comum, mas sim o endereço de memória de outra variável.

PONTEIROS - CONCEITO

- Por que utilizar ponteiros?
- Compartilhar dados entre diferentes funções.
- Evitar cópias desnecessárias.
- Criar estruturas de tamanho variável.
- Manipular memória dinamicamente.
- Implementar estruturas de dados como listas, filas e árvores.
- Desenvolver bibliotecas genéricas.

PONTEIROS - CONCEITO

Memória RAM, endereços e ponteiros

Cada posição da memória possui um endereço e armazena um valor.

Endereço	Conteúdo armazenado
0x1000	12
0x1004	25
0x1008	42
0x100C	0x1008
0x1010	7.5
0x1014	'A'
0x1018	0

O ponteiro armazena
o endereço 0x1008

Interpretação no programa

```
int a = 12;
```

```
int b = 25;
```

```
int idade = 42;
```

Variável comum:
armazena um valor

```
int *ponteiro = &idade;
```

Ponteiro:
armazena um endereço

```
float nota = 7.5f;
```

```
char letra = 'A';
```

```
int contador = 0;
```

A RAM é uma grande sequência de posições identificadas por endereços

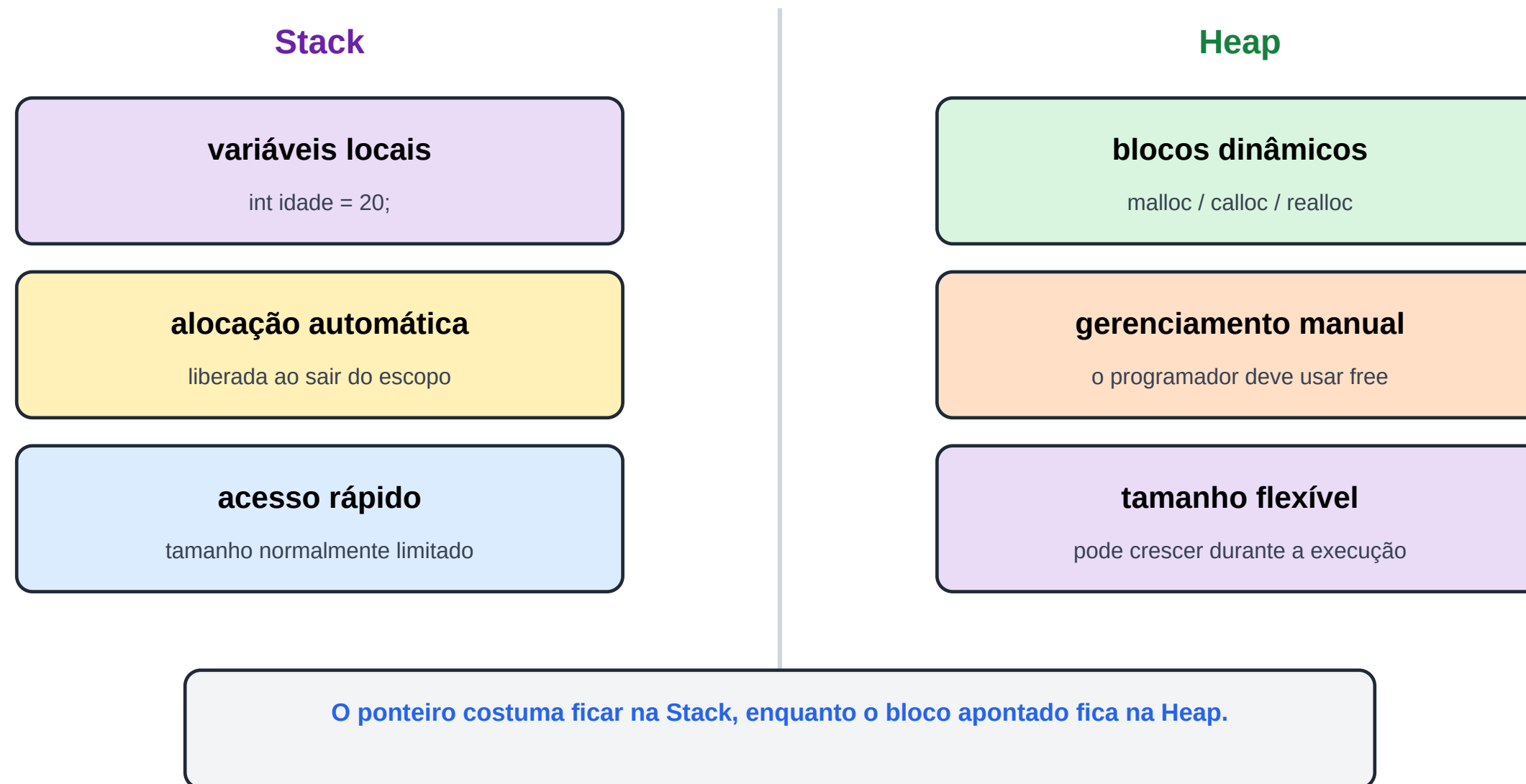
PONTEIROS - CONCEITO

- Uma das principais ideias ao utilizar ponteiros é evitar a duplicação de valores;
- o que evita problemas ao usar recursos escassos, como por exemplo um arquivo, um socket ou uma porta serial por exemplo;
- Além disso, ainda podemos tornar o programa mais dinâmico e otimizado, através da alocação dinâmica, podemos ter vetores de tamanhos específicos, que podem se expandir ou encolher, conforme a demanda;
- Além disso, podemos inclusive usar ponteiros para armazenar funções;

PONTEIROS - CONCEITO

Stack e Heap

As duas regiões armazenam dados, mas possuem ciclos de vida e formas de gerenciamento diferentes.



PONTEIROS - DECLARAÇÃO

- Antes de falarmos explicitamente sobre a declaração de ponteiros, devemos falar dos operadores principais para o trabalho com ponteiros;
- O operador `*` Acessa o valor, o conteúdo apontado pelo endereço do ponteiro
- O operador `&` Acessa o endereço de uma variável em memória

```
1
2 int main() {
3     int *p = NULL;
4 }
```

- Esse é o modo mais 'Seguro' para declaração de um ponteiro, pois garante que ele aponte para um endereço nulo (NULL);

PONTEIROS - DECLARAÇÃO

- Também é possível, declarar um ponteiro 'apontando' para uma variável específica, para assim tentar evitar a duplicação de valores

```
int main() {  
    int x = 10;  
    //dessa forma indicamos que o ponteiro P, aponta para o endereço da variável 'x'  
    int *p = &x;  
}
```

- Dessa forma toda operação executada com '*p', irá refletir no valor armazenado na variável 'x', pois ambas referenciam o mesmo endereço de memória

PONTEIROS - INICIALIZAÇÃO

- Ao utilizarmos a declaração `int *p = NULL;`, não podemos fazer qualquer operação ou acesso ao ponteiro, sem antes inicializa-lo devidamente.
- Isso pode causar um comportamento inesperado no programa, ou mesmo 'quebrá-lo', causando um problema na usabilidade e estabilidade do programa.
- A função 'malloc', deve ser utilizada pra incializar o ponteiro antes do uso;

```
1
2 #include <stdlib.h> //Necessário para a função 'malloc'
3
4 int main() {
5     int *p = NULL;
6
7     p = malloc(1 * sizeof(int)); // Aloca o ponteiro p para uma posição do tamanho do inteiro
8
9     if (p == NULL)
10    {
11        printf("Erro de alocação\n");
12        return 1;
13    }
14    return 0;
15 }
```

PONTEIROS - INICIALIZAÇÃO

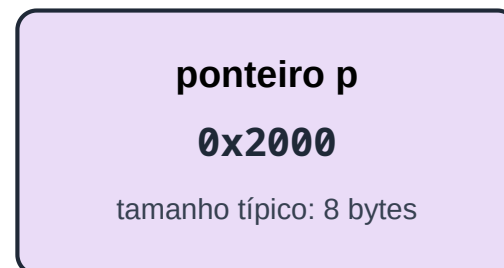
Alocação dinâmica com malloc

O ponteiro fica na Stack e armazena o endereço de um bloco reservado na Heap.

Alocação para int

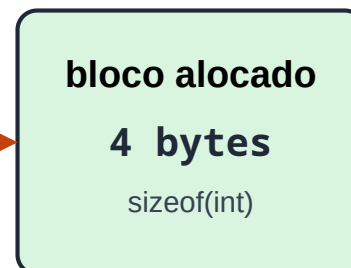
```
int *p;  
p = malloc(1 * sizeof(int));
```

Stack



endereço de p: 0x1000

Heap



endereço do bloco: 0x2000

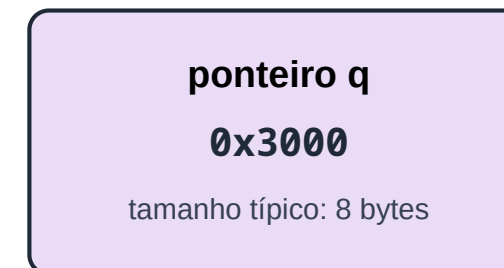
↓
Espaço suficiente para 1 inteiro

O bloco contém lixo de memória até receber um valor
***p = 42;**

Alocação para long

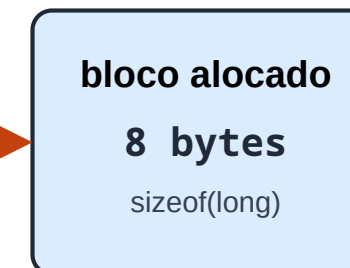
```
long *q;  
q = malloc(1 * sizeof(long));
```

Stack



endereço de q: 0x1010

Heap



endereço do bloco: 0x3000

↓
Espaço suficiente para 1 long

Em Linux 64 bits, long costuma ocupar 8 bytes
***q = 9000000L;**

O tamanho do ponteiro não determina o tamanho do bloco: malloc reserva exatamente a quantidade de bytes solicitada.

PONTEIROS - INICIALIZAÇÃO

- Além da função 'malloc', para inicializar ponteiros, podemos utilizar a função 'calloc'.
- A função 'calloc', essencialmente faz o mesmo que a função 'malloc', contudo, enquanto a função 'malloc', apenas aloca a memória inicializando o ponteiro, a função 'calloc' aloca a memória e inicializa cada uma das posições com o valor 0

```
1
2 int main(){
3     int *vetor = calloc(10, sizeof(int));
4 }
```


PONTEIROS - INICIALIZAÇÃO DE VETORES DINÂMICOS

- Assim como anteriormente alocamos um ponteiro para uma posição int, podemos alocar um ponteiro para 'N' posições de um inteiro por exemplo.

```
1
2 int main() {
3     int *p = NULL;
4     int n = 0;
5     printf("Insira o número de posições desejadas:\n")
6     scanf("%d",&n);
7     p = malloc(n * sizeof(int)); // Aloca o ponteiro p para n posições do tamanho do inteiro
8 }
```

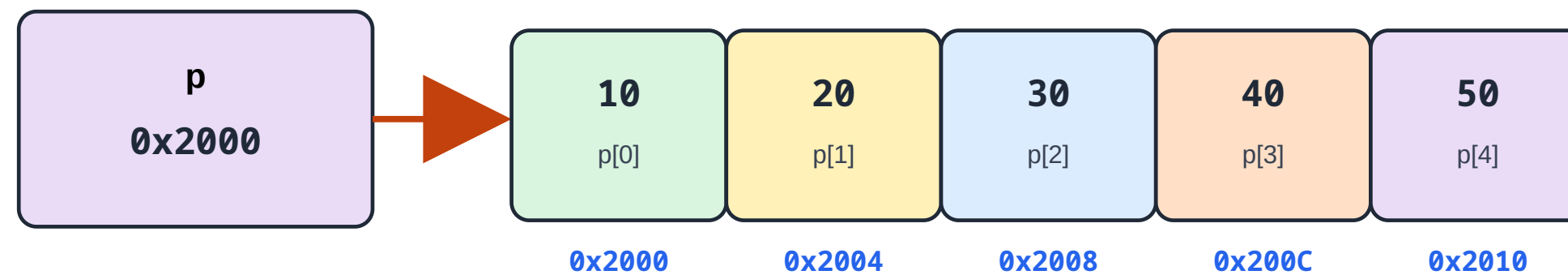
- Cada posição do ponteiro representa um número diferente.

PONTEIROS - INICIALIZAÇÃO DE VETORES DINÂMICOS

Vetor dinâmico em memória

Após a alocação, o ponteiro pode ser usado como um vetor comum.

```
int *p = malloc(5 * sizeof(int));
```



Aritmética de endereços: cada int avança 4 bytes

```
p[i] == *(p + i)
```

A notação de vetor é uma forma mais legível de acessar o bloco.

PONTEIROS - DEINICIALIZAÇÃO

- Diferente das outras variáveis, quando alocamos um vetor com 'malloc', somos obrigados a liberar a memória, do contrário isso vai gerar o que chamamos de *memory-leak*
- Na prática, o *memory-leak*, é de fato um 'vazamento' de memória, é um endereço ou uma faixa de endereços, que fica "perdida" na memória RAM, e que pode ser a causa de diversos tipos de erros, como uma "tela azul da morte", ou ainda um *kernel-panic*
- Como consequências mais comuns aos *memory-leaks* temos: aumento gradual do consumo de memória, degradação de desempenho, encerramento do processo pelo sistema operacional ou ainda falha por falta de memória

PONTEIROS - DEINICIALIZAÇÃO

```
1
2 int main() {
3     int *p = NULL;
4     int n = 0;
5     printf("Insira o número de posições desejadas:\n")
6     scanf("%d",&n);
7     p = malloc(n * sizeof(int)); // Aloca o ponteiro p para n posições do tamanho do inteiro
8     //...
9     free(p); // libera a memória alocada para o ponteiro 'p'
10 }
```

- Após o 'free', pode ser que o ponteiro ainda contenha um endereço salvo, contudo esse endereço não está mais em uso pelo programa, acessa-lo pode causar um comportamento inesperado

PONTEIROS - DEINICIALIZAÇÃO

Liberação de memória com free

free devolve à Heap o bloco reservado anteriormente.

```
free(p);
```

Stack

p

0x2000

endereço não é mais válido

Heap

bloco liberado

a memória pode ser reutilizada

```
free(p);
```

```
p = NULL;
```

PONTEIROS - REINICIALIZAÇÃO

- Em alguns casos pode ser necessário alocar mais posições para um ponteiro, já inicializado com o malloc, nesse caso podemos usar a função 'realloc'

```
1
2 int main() {
3     int *p = NULL;
4     int n = 0;
5     printf("Insira o número de posições desejadas:\n")
6     scanf("%d",&n);
7     p = malloc(n * sizeof(int)); // Aloca o ponteiro p para n posições do tamanho do inteiro
8     //...
9     int *temp = malloc(p,(n+x) * sizeof(int)); // Realoca o ponteiro p para n+x posições do tamanho do inteiro
10    if (temp != NULL)
11    {
12        p = temp;
13    }
14    //...
15    free(p); // libera a memória alocada para o ponteiro 'p'
```

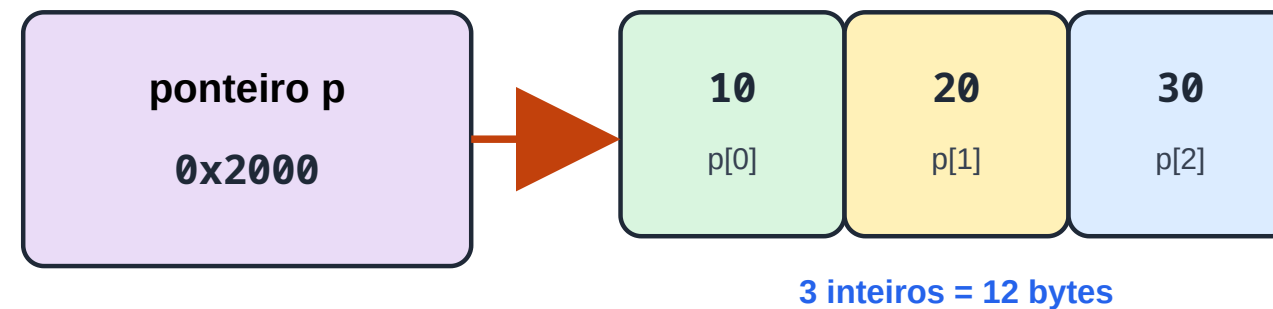
PONTEIROS - REINICIALIZAÇÃO

Redimensionamento de memória com realloc

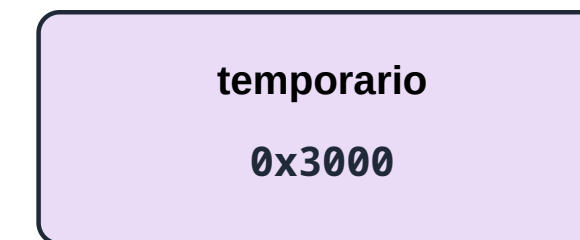
realloc tenta alterar o tamanho do bloco já existente e preserva os dados anteriores.

```
int *temporario = realloc(p, 5 * sizeof(int));
```

Antes do realloc



Possibilidade B: muda de endereço



Possibilidade A: cresce no mesmo endereço



O endereço permanece 0x2000 e os dados antigos são preservados

Os dados são copiados para um novo bloco e o antigo é liberado

Não atribua realloc diretamente a p
`if (temporario != NULL) p = temporario;`

As novas posições não são inicializadas: elas contêm valores indeterminados.

PONTEIROS - GENÉRICOS

- Além dos ponteiros definidos como, int, long, float, podemos ter ainda a necessidade de criarmos ponteiros genéricos, para dados customizados do usuário
- Um ponteiro genérico (void *) é um ponteiro capaz de armazenar o endereço de qualquer tipo de dado. Diferentemente dos demais ponteiros, ele não está associado a um tipo específico, sendo amplamente utilizado em funções genéricas, bibliotecas da linguagem C e em mecanismos de alocação dinâmica de memória, como malloc(), calloc() e realloc().
- Como um void * não conhece o tipo do dado armazenado, não é possível acessar diretamente seu conteúdo. Antes de utilizar o valor apontado, é necessário realizar uma conversão de tipo (cast) para o tipo correto.

PONTEIROS - GENÉRICOS

```
1
2 #include <stdio.h>
3 typedef struct {
4     char nome[30];
5     int idade;
6 } Pessoa;
7
8 int main() {
9     Pessoa aluno = {
10         "Maria",
11         20
12     };
13     void *ptr = &aluno;
14     Pessoa *pessoa = (Pessoa *) ptr;
15     printf("Nome : %s\n", pessoa->nome);
```

PONTEIROS PARA FUNÇÕES

- Assim como é possível armazenar o endereço de uma variável em um ponteiro, também é possível armazenar o endereço de uma função. Os ponteiros para funções permitem selecionar dinamicamente qual função será executada durante a execução do programa, sendo amplamente utilizados na implementação de menus, máquinas de estados, tabelas de despacho (dispatch tables), bibliotecas e algoritmos genéricos.
- A declaração de um ponteiro para função deve especificar o tipo de retorno e os tipos dos parâmetros da função que ele poderá armazenar.

PONTEIROS PARA FUNÇÕES

```
1
2 #include <stdio.h>
3
4 int soma(int a, int b) {
5     return a + b;
6 }
7
8 int subtracao(int a, int b) {
9     return a - b;
10 }
11
12 int main() {
13     /* Ponteiro para função */
14     int (*operacao)(int, int);
15     operacao = soma;
```


PONTEIROS PARA FUNÇÕES

- Nesse exemplo, o ponteiro operacao pode armazenar o endereço de qualquer função que receba dois parâmetros do tipo int e retorne um int. Alterando o endereço armazenado no ponteiro, o programa passa a executar uma função diferente sem modificar sua lógica principal.

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br